

Project 주간보고서

학부(전공)/학과	응용소프트웨어공학과	작성일	2026. 05. 31 (13주차)
팀명	이음	작성자 성명	홍효정 (서명)
프로젝트명	캡스톤디자인II	담당(지도)교수	김삼문

항목	내용		비고
이번 주 수행 내용	공통	● 기능 완성 및 프로젝트 마무리 ● 프로젝트 형상 관리	회의록 첨부
	홍효정 (팀장)	● frontend 폴더 내의 코드 정리 ● 추가 수정사항 점검 및 전달	회의록 첨부
	박정열	● 예외 및 오류 처리 확인 ● 감시기능, 감시 OCR기능 점검 ● backend 폴더내의 코드 정리	-
다음 주 수행 계획	공통	● 프로젝트 발표	-
	홍효정 (팀장)	● ppt, 및 영상 제작(개인), 영상 편집(공통)	-
	박정열	● ppt 및 영상 제작(개인,공통)	-
기타			
담당교수 지시사항			확인 날인
프로젝트 지도교수 지시사항	김삼문		확인 날인
			

※회의록 및 회의 증빙 사진 첨부

Project 팀 회의록

		과목명	캡스톤디자인II
회의 주제	프로젝트 마무리		
학부(전공)/학과	응용소프트웨어공학과		
회의 일시	2026. 05. 31 15:57 ~ 16:27	회의 장소	온라인 회의
참석자	팀장 홍효정 (서명)	팀원 박정열 (서명)	
회의 내용			
주요 안건	1. 감시 기능 및 감시 문서 작성 기능 점검 2. 기능 요구사항 비교 점검 및 추가 요구사항 전달 3. 프로젝트 형상관리 및 마무리		
● 이전 진행 내용 - AI OCR 사진 데이터 추출 Split-View 출력 - AI OCR 문서 데이터 추출 Split-View 출력 - 추출된 데이터 기반 자동 문서화 로직 설계 ● 안건 1 감시 기능 및 감시 문서 작성 기능 점검 <pre>#main.py 감시 관련 - (해당 코드 탑재) ● 기능 3,4 ● 모델로드 상단에 백그라운드 실시간 분석 엔진, ● 실시간 감지 상태 관리 및 중복 방지 설정 # [기능 3] YOLOv10 객체 탐지 및 안전모 미착용 집중 분석 파이프라인 @app.post("/api/analyze-image") async def analyze_uploaded_image(company_id: str = Form(...), camera_id: int = Form(1), file: UploadFile = File(...)): try: # 1. 파일 읽기 및 변환 image_bytes = await file.read() nparr = np.frombuffer(image_bytes, np.uint8) frame = cv2.imdecode(nparr, cv2.IMREAD_COLOR) # 2. YOLOv10 분석 실행 (사람과 헬멧을 확실히 잡기 위해 conf 조절) results = yolo_model.predict(frame, conf=0.25, verbose=False) persons = [] # 사람 좌표 helmets = [] # 안전모 좌표 detections = [] # 로그용 전체 감지 결과 is_violation = False</pre>			

```

annotated_frame = frame.copy()

# 3. 1차 스캔: 모든 사람(6)과 안전모(0), 직접 미착용(7) 식별
for r in results:
    for box in r.bboxes:
        cls_id = int(box.cls[0])
        conf = float(box.conf[0])
        label = yolo_model.names[cls_id]
        coords = box.xyxy[0].cpu().numpy().astype(int)

        detections.append({"label": label, "confidence": conf, "cls_id": cls_id})

    if cls_id == 6: # Person
        persons.append(coords)
    elif cls_id == 0: # Helmet
        helmets.append(coords)
    elif cls_id == 7: # 직접 감지된 no_helmet
        is_violation = True
        # 직접 감지된 미착용자 표시
        cv2.rectangle(annotated_frame, (coords[0], coords[1]), (coords[2],
coords[3]), (0, 0, 255), 3)
        cv2.putText(annotated_frame, "No Helmet", (coords[0], coords[1]-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

# 4. 2차 스캔: 역발상 로직 (사람 박스 안에 헬멧이 없는 경우)
for p_box in persons:
    px1, py1, px2, py2 = p_box
    has_helmet = False

    for h_box in helmets:
        hx1, hy1, hx2, hy2 = h_box
        # 헬멧 박스가 사람 박스 가로 폭 안에 있고, 사람의 상단 영역에 걸쳐있는지 확인
        # (단순 겹침 판단)
        if not (hx2 < px1 or hx1 > px2 or hy2 < py1 or hy1 > py2):
            has_helmet = True
            break

    if not has_helmet:
        is_violation = True
        # 안전모 미착용자로 판단된 사람(Person) 박스 표시
        cv2.rectangle(annotated_frame, (px1, py1), (px2, py2), (0, 0, 255), 3)
        cv2.putText(annotated_frame, "No Helmet Detected", (px1, py1-10),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0, 0, 255), 2)

```

```
# 5. 위반 발생 시 처리
```

```
if is_violation:
```

```
    current_time = datetime.now()
```

```
    if not os.path.exists('temp'): os.makedirs('temp')
```

```
    filename = f"helmet_violation_{current_time.strftime('%Y%m%d_%H%M%S')}.jpg"
```

```
    save_path = f"temp/{filename}"
```

```
    cv2.imwrite(save_path, annotated_frame)
```

```
    # Supabase Storage 업로드
```

```
    with open(save_path, 'rb') as f:
```

```
        supabase.storage.from_("snapshots").upload(filename, f)
```

```
    snapshot_url = supabase.storage.from_("snapshots").get_public_url(filename)
```

```
    # Supabase DB 저장 (safety_logs)
```

```
    supabase.table("safety_logs").insert({
```

```
        "company_id": company_id,
```

```
        "camera_id": int(camera_id),
```

```
        "violation_type": "안전모 미착용",
```

```
        "snapshot_url": snapshot_url,
```

```
        "detected_at": current_time.isoformat()
```

```
    }).execute()
```

```
    return {
```

```
        "status": "violation",
```

```
        "message": "경고: 안전모 미착용 작업자가 감지되었습니다.",
```

```
        "detections": detections,
```

```
        "image_url": snapshot_url
```

```
    }
```

```
# 6. 위반 없음
```

```
return {
```

```
    "status": "safe",
```

```
    "message": "모든 작업자가 안전모를 착용 중입니다.",
```

```
    "detections": detections
```

```
}
```

```
except Exception as e:
```

```
    print(f"이미지 분석 오류: {e}")
```

```
    raise HTTPException(status_code=500, detail=f"분석 중 오류 발생: {str(e)}")
```

```
# [기능 4] 카메라연결
```

```

@app.post("/camera/connect")
async def connect_camera(req: CameraConnectRequest):

    company_id = req.company_id
    # 1. 스트림 URL 판단
    stream_url = "0"
    if "[폰]" in req.name:
        ip_match = re.search(r'\\((.*?)\\)', req.location)
        if ip_match:
            ip_address = ip_match.group(1).strip()
            stream_url = f"http://{ip_address}/video" if not ip_address.startswith("http")
        else f"{ip_address}/video"

    cam_id_key = int(req.camera_id)
    company_id = req.company_id

    # --- [추가] DB에 카메라 정보 등록 (UPSERT 로직) ---
    try:
        # cameras 테이블에 이 카메라 ID가 있는지 확인하고, 없으면 생성/있으면 수정합니다.
        supabase.table("cameras").upsert({
            "id": cam_id_key,
            "company_id": company_id, # UUID 형식 그대로 들어감
            "location_name": req.location,
            "is_active": True,
            "stream_url": stream_url
        }).execute()
        print(f"✔ [DB] 카메라 {cam_id_key} 정보 등록/업데이트 완료")
    except Exception as e:
        print(f"✖ [DB] 카메라 등록 실패: {e}")
        # 카메라 등록에 실패하면 외래키 제약조건 때문에 로그 저장이 안 되므로 에러를 던지는
        # 것이 좋습니다.
        raise HTTPException(status_code=500, detail="카메라를 DB에 등록할 수 없습니다.")
    # -----

    # 2. 기존 실행 중인 동일 카메라가 있다면 중지
    if cam_id_key in active_streams:
        active_streams[cam_id_key] = False
        await asyncio.sleep(0.5) # 종료 대기

    # 3. 백그라운드 태스크로 분석 시작
    active_streams[cam_id_key] = True
    asyncio.create_task(start_realtime_monitoring(cam_id_key, company_id, stream_url))

    return {

```

```

        "status": "online",
        "message": f"카메라 {cam_id_key} 실시간 감시 모드가 시작되었습니다.",
        "stream_url": stream_url
    }

```

cctv프론트 창에 데이터 전달 - (해당 코드 탑재)

- 기능 10,11,

[기능 10] AI 위험 감지 로그 리스트 불러오기

@app.get("/api/safety-logs")

def get_safety_logs(company_id: str):

try:

safety_logs와 cameras 테이블 조인 (카메라 위치 획득)

response = supabase.table("safety_logs") \

.select("""

id,

detected_at,

camera_id,

violation_type,

status,

snapshot_url,

cameras (location_name)

""") \

.eq("company_id", company_id) \

.order("detected_at", desc=True) \

.execute()

return {"logs": response.data}

except Exception as e:

print(f"로그 불러오기 에러: {str(e)}")

raise HTTPException(status_code=500, detail=f"데이터 불러오기 실패: {str(e)}")

[기능 11] AI 위험 감지 로그 상태 업데이트

@app.patch("/api/safety-logs/{log_id}")

async def update_safety_log(log_id: int, req: UpdateLogRequest):

try:

DB의 status 컬럼 업데이트

response = supabase.table("safety_logs").update({

"status": req.status,

"admin_memo": req.admin_memo

}).eq("id", log_id).execute()

if not response.data:

raise HTTPException(status_code=404, detail="해당 로그를 찾을 수 없습니다.")

```
return {"message": "상태가 성공적으로 업데이트되었습니다.", "data": response.data}
```

```
except Exception as e:
```

```
    print(f"상태 업데이트 에러: {str(e)}")
```

```
    raise HTTPException(status_code=500, detail=f"업데이트 실패: {str(e)}")
```

cctv 문서 생성 - (해당코드 탑재)

- 기능 12

[기능 12] AI 안전 보고서 생성 및 다운로드 API (PDF 생성 및 데이터 심층 분석)

```
@app.get("/api/download-safety-report")
```

```
async def download_safety_report(company_id: str, start_date: str = None, end_date: str = None):
```

```
    try:
```

```
        # 1. Supabase에서 안전 로그 데이터 및 카메라 위치 가져오기
```

```
        query = supabase.table("safety_logs").select("""
```

```
            id,
```

```
            detected_at,
```

```
            violation_type,
```

```
            status,
```

```
            admin_memo,
```

```
            snapshot_url,
```

```
            cameras ( location_name )
```

```
        """).eq("company_id", company_id)
```

```
        if start_date:
```

```
            query = query.gte("detected_at", f"{start_date} 00:00:00")
```

```
        if end_date:
```

```
            query = query.lte("detected_at", f"{end_date} 23:59:59")
```

```
        response = query.order("detected_at", desc=True).execute()
```

```
        logs = response.data if response.data else []
```

```
        if not logs:
```

```
            raise HTTPException(status_code=404, detail="보고서를 생성할 안전 로그 데이터가 없습니다.")
```

```
        # 기본 요약 통계 계산
```

```
        total_count = len(logs)
```

```
        confirmed_count = sum(1 for l in logs if l.get("status") in ["경고 확정", "경고확정"])
```

```
        unchecked_count = sum(1 for l in logs if l.get("status") in ["UNCHECKED", "미확인"])
```

```
        resolved_count = total_count - unchecked_count
```

```
        # 2. OpenAI 가공용 로그 요약문 작성 (최근 15개 중심 심층 요약)
```

```

log_summary_text = ""
for log in logs[:15]:
    loc = log.get("cameras", {}).get("location_name", "미확인") if log.get("cameras")
else "미확인"
    log_summary_text += f"- 로그번호: {log['id']}, 시간: {log['detected_at']}, 위치: {loc}, 종류: {log['violation_type']}, 상태: {log['status']}, 조치메모: {log['admin_memo']} or '없음'\n"

# 3. OpenAI GPT-4o-mini 에 안전 종합 총평 및 분석 요청
prompt = f"""
당신은 산업 현장의 인공지능 안전 분석관(AI Safety Officer)입니다.
최근 물류창고 내부에서 감지된 '안전 위반(안전모 미착용 등)' 로그 통계와 내역 데이터를
기반으로, 종합 안전 분석 보고서 본문을 작성해 주세요.

[종합 데이터 통계]
- 전체 감지 건수: {total_count}건
- 관리자 경고 확정 건수: {confirmed_count}건
- 미확인 대기 건수: {unchecked_count}건

[최근 발생한 주요 위반 사례 정보]
{log_summary_text}

[보고서 작성 필수 규칙]
1. 반드시 전문적이고 객관적인 비즈니스 톤앤매너의 한국어로 작성하세요.
2. HTML 구조 태그(예: <h3>, <p>, <ul>, <li>, <strong>)만을 활용해서 세련된 텍스트 서
식을 만드세요. 코드 블록(``html)은 절대 포함하지 말고 순수 태그 텍스트 내용만 출력하세요.
3. 하위 3개 세션을 명확히 구분하여 심도 깊은 분석을 제공하세요:
    - 1) 현장 안전 보건 관리 총평 (현재 위험 징후 및 검수율에 대한 평가)
    - 2) 위험 요소 분석 및 취약 구역 패턴 파악 (사사로운 발생 구역 정보 종합 분석)
    - 3) 현장 안전 강화를 위한 긴급 대응 및 예방 대책 제안
"""

ai_response = await openai_client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "산업 안전 진단 전문가로서 고품질 보고서 내용을
구조화된 HTML 서식으로 작성하는 역할입니다."},
        {"role": "user", "content": prompt}
    ],
    temperature=0.7
)
ai_analysis_html = ai_response.choices[0].message.content
ai_analysis_html = ai_analysis_html.replace("`html", "").replace("```", "").strip()

```



```

# 4. 전체 HTML 템플릿 + WeasyPrint 전용 CSS 스타일링
log_rows_html = ""
for log in logs:
    loc = log.get("cameras", {}).get("location_name", "미확인") if log.get("cameras")
else "미확인"
    img_src = log.get("snapshot_url", "")
    img_html = f'' if img_src else '<span
style="color:#aaa;">이미지 없음</span>'
    date_str = log['detected_at'][:19].replace("T", " ")
    memo_str = log.get("admin_memo") or "-"

    log_rows_html += f"""
<tr>
    <td style="text-align: center; font-weight: bold;">{log['id']}</td>
    <td style="text-align: center; white-space: nowrap;">{date_str}</td>
    <td>{loc}</td>
    <td
        style="text-align: center;"><span
badge-danger">{log['violation_type']}</span></td>
    <td style="text-align: center; font-weight: bold; color: {'#ef4444' if
log['status'] in ['경고 확정', '경고확정'] else '#64748b'};">{log['status']}</td>
    <td>{memo_str}</td>
    <td style="text-align: center;">{img_html}</td>
</tr>
"""

current_report_time = datetime.now().strftime("%Y-%m-%d %H:%M")

html_report_template = f"""
<!DOCTYPE html>
<html>
<head>
    <meta charset="utf-8">
    <title>SafeLogiX AI Safety Report</title>
    <style>
        @page {{
            size: A4;
            margin: 20mm 15mm 20mm 15mm;
            @bottom-right {{
                content: counter(page) " / " counter(pages);
                font-size: 9pt;
                color: #94a3b8;
            }}
            @bottom-left {{
                content: "SafeLogiX AI 자동화 시스템 안전 진단 보고서";

```

```
        font-size: 9pt;
        color: #94a3b8;
    }}
}}
body {{
    font-family: 'Malgun Gothic', 'Apple SD Gothic Neo', sans-serif;
    color: #1e293b;
    line-height: 1.6;
    font-size: 10pt;
    margin: 0;
    padding: 0;
}}
.header-container {{
    border-bottom: 3px solid #0f172a;
    padding-bottom: 12px;
    margin-bottom: 30px;
}}
.header-container h1 {{
    font-size: 26pt;
    color: #0f172a;
    margin: 0 0 8px 0;
    font-weight: 800;
    letter-spacing: -1px;
}}
.header-container .meta-info {{
    font-size: 10pt;
    color: #64748b;
    text-align: right;
}}
.section-header {{
    font-size: 14pt;
    color: #0f172a;
    border-left: 5px solid #ef4444;
    padding-left: 10px;
    margin-top: 30px;
    margin-bottom: 15px;
    font-weight: bold;
}}
.summary-box-table {{
    width: 100%;
    border-collapse: collapse;
    margin-bottom: 25px;
}}
.summary-box-table th, .summary-box-table td {{
```

```
        border: 1px solid #cbd5e1;
        padding: 12px;
        text-align: center;
    }}
    .summary-box-table th {{
        background-color: #f8fafc;
        color: #475569;
        font-weight: bold;
    }}
    .ai-generated-content {{
        background-color: #fafafa;
        border: 1px solid #e2e8f0;
        border-radius: 8px;
        padding: 20px;
        margin-bottom: 30px;
    }}
    .ai-generated-content h3 {{
        font-size: 12pt;
        color: #1e293b;
        margin-top: 15px;
        margin-bottom: 8px;
        border-bottom: 1px solid #e2e8f0;
        padding-bottom: 6px;
    }}
    .ai-generated-content ul, .ai-generated-content ol {{
        margin-top: 5px;
        padding-left: 20px;
    }}
    .ai-generated-content li {{
        margin-bottom: 4px;
    }}
    .main-log-table {{
        width: 100%;
        border-collapse: collapse;
        margin-top: 15px;
        font-size: 9pt;
    }}
    .main-log-table th {{
        background-color: #0f172a;
        color: #ffffff;
        font-weight: bold;
        text-align: center;
        padding: 10px 8px;
        border: 1px solid #0f172a;
```

```
    }}
    .main-log-table td {{
        border: 1px solid #e2e8f0;
        padding: 8px 8px;
        vertical-align: middle;
    }}
    .main-log-table tr:nth-child(even) {{
        background-color: #f8fafc;
    }}
    .report-img {{
        max-width: 110px;
        max-height: 75px;
        border-radius: 4px;
        border: 1px solid #cbd5e1;
        box-shadow: 0 1px 3px rgba(0,0,0,0.05);
    }}
    .badge {{
        display: inline-block;
        padding: 3px 7px;
        font-size: 8pt;
        font-weight: bold;
        border-radius: 4px;
        color: #ffffff;
    }}
    .badge-danger {{
        background-color: #ef4444;
    }}
</style>
</head>
<body>
    <div class="header-container">
        <h1>AI 자율형 안전보건 진단 보고서</h1>
        <div class="meta-info">출력일시: {current_report_time} | 사업장(회사) 식별코
드: {company_id}</div>
    </div>

    <div class="section-header">1. 실시간 모니터링 안전 위반 요약 통계</div>
    <table class="summary-box-table">
        <thead>
            <tr>
                <th style="width: 25%;">총 위험 감지 건수</th>
                <th style="width: 25%;">관리자 확정 건수</th>
                <th style="width: 25%;">미확인 대기 건수</th>
                <th style="width: 25%;">안전 검수 조치율</th>
            </tr>
        </thead>
    </table>
</body>
</html>
```

```

        </tr>
    </thead>
    <tbody>
        <tr>
            <td style="font-size: 15pt; font-weight: bold; color: #ef4444;">{total_count} 건</td>
            <td style="font-size: 13pt; font-weight: bold; color: #2563eb;">{confirmed_count} 건</td>
            <td style="font-size: 13pt; font-weight: bold; color: #64748b;">{unchecked_count} 건</td>
            <td style="font-size: 15pt; font-weight: bold; color: #16a34a;">
                {f"{{resolved_count / total_count * 100}}.1f}%" if total_count >
0 else "0.0%"}
            </td>
        </tr>
    </tbody>
</table>

<div class="section-header">2. AI 종합 심층 분석 리포트 (gpt-4o-mini)</div>
<div class="ai-generated-content">
    {ai_analysis_html}
</div>

<div style="page-break-before: always;"></div>

<div class="section-header">3. 전체 안전 위험 감지 및 조치 내역 (상세 로그)</div>
<table class="main-log-table">
    <thead>
        <tr>
            <th style="width: 6%;">No</th>
            <th style="width: 17%;">감지 시간</th>
            <th style="width: 15%;">카메라 위치</th>
            <th style="width: 15%;">위험 분류</th>
            <th style="width: 12%;">상태</th>
            <th style="width: 19%;">조치사항 및 메모</th>
            <th style="width: 16%;">현장 현황 스냅샷</th>
        </tr>
    </thead>
    <tbody>
        {log_rows_html}
    </tbody>
</table>
</body>
</html>

```

```

# 5. WeasyPrint를 활용한 PDF 바이트 파일 컴파일
pdf_stream = io.BytesIO()
HTML(string=html_report_template).write_pdf(pdf_stream)
pdf_stream.seek(0)

formatted_date = datetime.now().strftime("%Y%m%d_%H%M")
download_filename = f"AI_Safety_Report_{formatted_date}.pdf"

return StreamingResponse(
    pdf_stream,
    media_type="application/pdf",
    headers={"Content-Disposition": f'attachment; filename="{download_filename}"'}
)

except Exception as e:
    print(f"보고서 다운로드 실패 오류: {str(e)}")
    raise HTTPException(status_code=500, detail=f"AI 안전 보고서 빌드 중 내부 오류가 발생했습니다: {str(e)}")

```

● 안전 2 기능 요구사항 비교 점검 및 추가 요구사항 전달

The screenshot displays the SafeLogiX AI Threat Detection interface. On the left, there is a sidebar with navigation options like '물류 현황' (Logistics Status) and 'CCTV 기록' (CCTV Record). The main area shows a table titled 'AI 위험 감지 로그' (AI Threat Detection Log) with columns: No, 감지 시간 (Detection Time), 카메라 위치 (Camera Location), 위험 분류 (Threat Classification), and 상태 (Status). A single entry is visible with No 99, 감지 시간 2026. 05. 31. 오후 02:04:38, 카메라 위치 스마트폰 (192.168.200.100:4747), 위험 분류 안전모 미착용 (Safety helmet not worn), and 상태 경고 확정 (Warning confirmed). To the right, there is a video feed showing a person in a warehouse, and a sidebar with controls for the video feed and a '위험 기록 상세 검수' (Threat Record Detailed Check) section. The sidebar also includes buttons for '미확인' (Unconfirmed), '경고 확정' (Warning confirmed), and '오탈지/정상' (Abnormal/Normal).

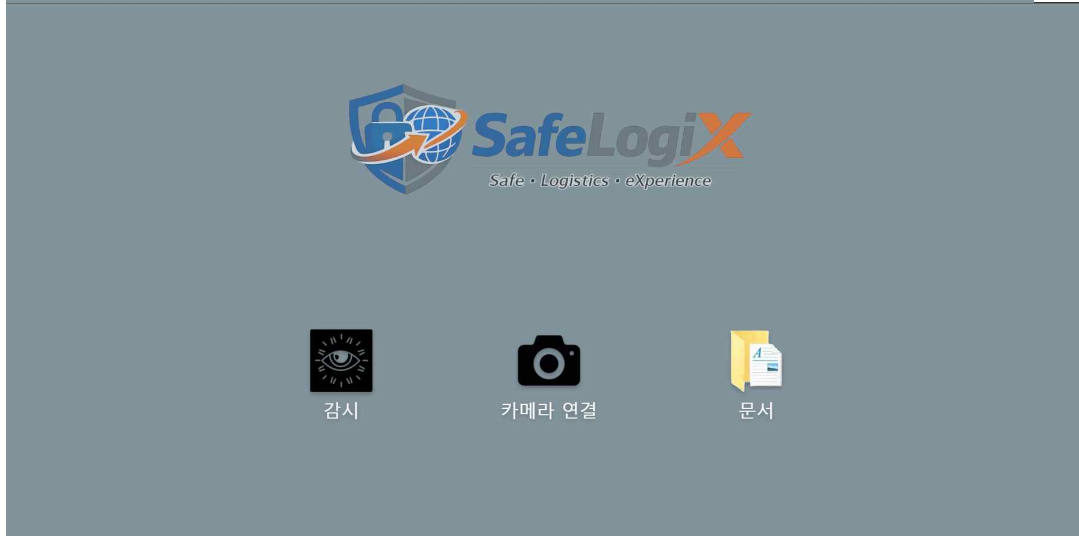
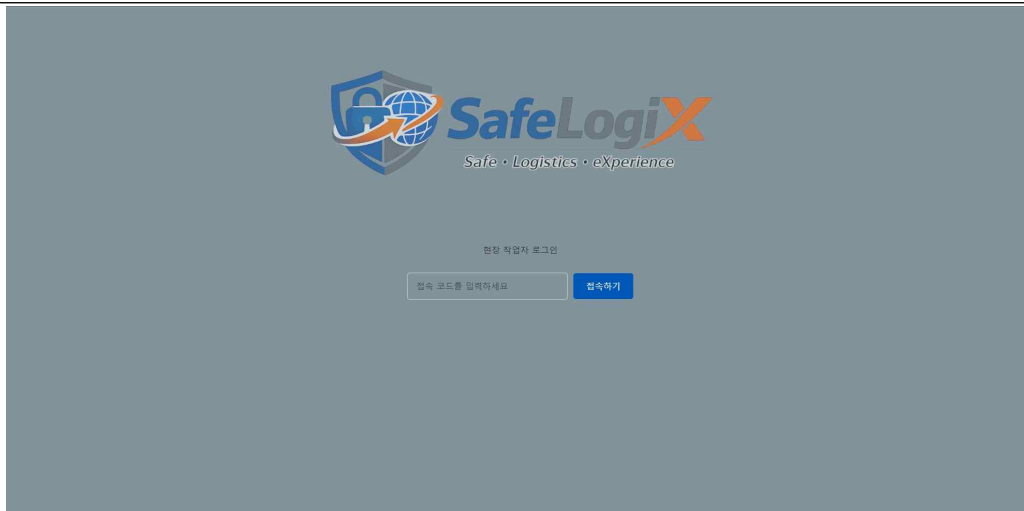
요청사항

현재 위험감지 로그 테이블 크기가 데이터랑 맞지 않아서

테이블과 데이터가 올바르게 정렬 되어 있지 않음

cctv.tsx파일 내 혹시 인라인 스타일 있는지 검토후 있다면 삭제요망

● 안전 3 프로젝트 형상관리 및 마무리



SafeLogiX

- 문류 현황
- CCTV 기록

AI 위험 감지 로그

No	감지 시간	카메라 위치	위험 분류	상태
99	2026. 05. 31. 오후 02:04:38	스마트폰 (192.168.200.100:4747)	안전모 미착용	경고 확정

위험 기록 상세 검수

감지내용: 안전모 미착용
발생시간: 2026. 05. 31. 오후 02:04:38

관리자 검수 상태

조지사항 및 메모

작업자에게 경고 조치함.

물류 현황

CCTV 기록

No	날짜	품목명	구분	수량	업체명
1	2026. 5. 24.	A	업고	100	ABC컴파니
2	2026. 5. 24.	기계식 키보드	업고	21	ABC 주식회사
3	2026. 5. 24.	A4 노트북	업고	49	ABC 주식회사
4	2026. 5. 24.	0.5 mm 펜	업고	100	ABC 주식회사

카메라 연결

← 돌아가기

활형 스마트본

카메라 이름

설치 위치

+ 추가

이름	위치	상태
테스트 촬영	사무실 (내 PC)	오프라인



카메라를 선택하세요

오른쪽 목록에서 선택

선택 완료

- 차기 회의 주제 안내
- 프로젝트 발표

[별첨] 회의 증빙 사진

